CAPTCHALAB AI

How to read this document

“Assessment requirement” quotes or paraphrases the supplied assignment brief. “Implemented” describes the code found in the packaged source. “Reason” explains the engineering choice. “Limit / not claimed” prevents the project from claiming capabilities that the code does not provide.

Contents and document purpose

- 1. Executive summary and project scope
- 2. What the assessment document actually requires
- 3. Architecture and end-to-end workflows
- 4. Module 01 — URL-driven Playwright recognition
- 5. Module 02 — Synthetic CAPTCHA identification and solving
- 6. Module 03 — Official provider token acquisition and verification
- 7. Module 04 — CSV import, benchmark, metrics and dashboard
- 8. Function-by-function code reference
- 9. Assessment traceability matrix and scoring weights
- 10. Bonus feature comparison
- 11. Why the application deliberately does not do certain things
- 12. Technology, cost, configuration and security
- 13. Dataset packaging, reproducibility and test evidence
- 14. Demonstration plan and honest claims to make

1. Executive summary and project scope

CaptchaLab AI is a local Python web application. It combines a FastAPI backend, a lightweight Jinja2/JavaScript web interface, Playwright Chromium for authorized webpage inspection, OpenCV/NumPy/Pillow for local synthetic CAPTCHA processing, official provider Siteverify integrations, and SQLite for attempts and imported reference records.

The application is intentionally divided into four functional modules. Module 01 recognizes CAPTCHA provider indicators on one authorized URL. Module 02 identifies and solves three candidate-controlled synthetic challenge families. Module 03 obtains tokens from official provider test widgets and verifies them server-side. Module 04 imports the interviewer's 200-row website CSV, runs the synthetic batch benchmark, and displays reporting/metrics.

Most important scope distinction

The supplied PDF explicitly says contact-form automation and bulk website submissions are not part of this assignment. It also limits testing to synthetic CAPTCHAs, local demonstrations, candidate-owned websites, or official provider test environments. Therefore, the 200 CSV URLs are imported as reference metadata; the application does not automatically crawl all 200 sites, submit their forms, harvest tokens, or bypass their production CAPTCHA protections.

2. What the assessment document actually requires

The source document is "Assignment 01: CAPTCHA Solving & Verification Tool", a technical candidate assessment for an AI Automation / Full-Stack Developer role. It suggests 6–8 hours and evaluates automation design, AI/computer vision or API integration, secure token handling, reliability, and reporting.

Brief section	Requirement in plain language	What it is testing

Brief section	Requirement in plain language	What it is testing
1. Project Overview	Build a standalone application that recognizes and solves supported CAPTCHA challenges in an authorized test environment and integrates an official verification flow.	A working, runnable tool not only a slide deck.
2.A Recognition	Identify the presented challenge type; distinguish supported from unsupported; recognize Google reCAPTCHA, hCaptcha and Cloudflare Turnstile in a controlled test integration.	Detection and classification.
2.B Solving Engine	Implement at least one custom OCR/CV/ML method for synthetic or candidate-controlled CAPTCHA data, or an approved low-cost API; report solution and confidence where meaningful.	A genuine solving workflow.
2.C Verification Tokens	Obtain official test tokens and validate them server-side; handle invalid, expired, reused and missing tokens; return structured responses without exposing secrets/tokens in logs.	Secure provider integration.
2.D Interface & Reporting	Allow upload/select, process and display results; report attempts, success/failure, processing time, challenge type, method/provider and estimated cost when third-party services are used.	Usability and observability.
3. Technical Expectations	Use a suitable stack; prefer modular architecture, secure configuration, sensible timeouts and readable docs.	Engineering quality.
5. Required Deliverables	Source/setup/env docs; working demo; one custom solver and one official token verification flow; 3–5 min video; short architecture and cost explanation.	Reproducibility and proof.
6. Bonus Features	Multiple challenge types/providers; fallback; batch and accuracy stats; reusable API endpoint.	Extra breadth and integration.
7. Scope & Acceptance	Test only synthetic, local, owned or official provider environments; no unrelated-site bypass, token harvesting, or unsolicited submissions. Reviewer must run the app and inspect actual functionality.	Authorization and runnable proof.

3. Architecture and end-to-end workflows

The architecture separates three different types of work that should not be confused: page/provider recognition, custom synthetic solving, and official provider token verification. CSV/metrics support these workflows but do not replace them.

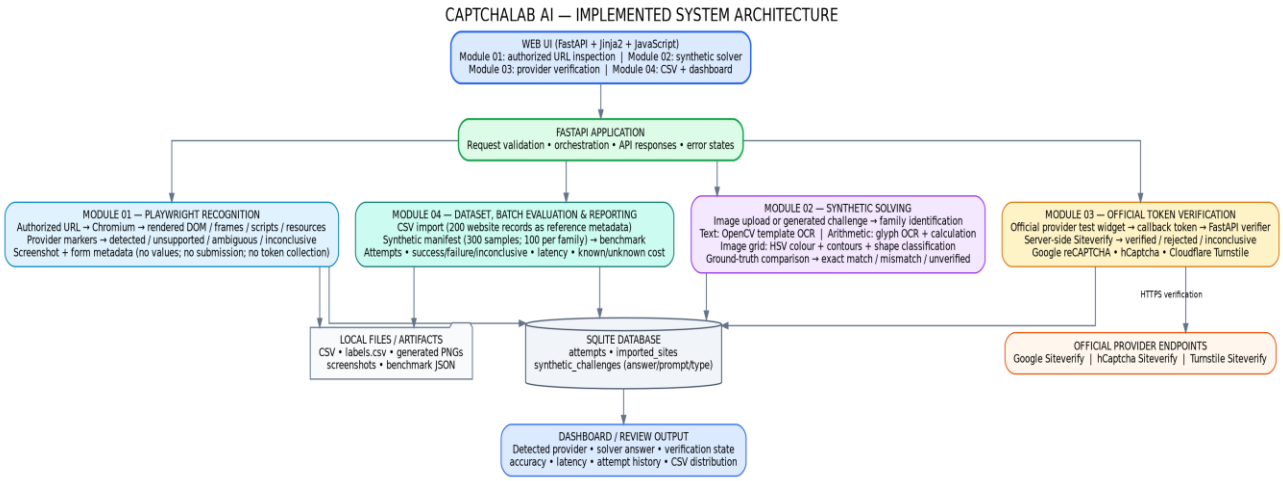


Figure 1. Implementation architecture derived from the packaged source tree.

3.1 Workflow A: Inspect an authorized webpage

1. The user enters one URL in Module 01. The backend validates the scheme and hostname against ALLOWED_TARGET_HOSTS.
2. Playwright launches its managed Chromium browser (headless by default; the UI has “Show Chromium window” for a visible inspection).
3. The browser loads the page and lets page JavaScript render. The inspector records frames, script/resource URLs, relevant request failures, HTTP errors and JavaScript/frame diagnostics.
4. The inspector captures form metadata only: method, sanitized action URL, number of fields and field types. It does not read field values or submit forms.
5. The recognition rules look for supported provider markers and known unsupported markers. The result can be a provider, none, unknown, multiple/ambiguous, or an inconclusive inspection.
6. A screenshot and a structured report are returned where possible. The attempt is saved for reporting.

3.2 Workflow B: Identify and solve a synthetic challenge

7. The user either asks the app to generate a challenge or uploads a locally owned/synthetic image.
8. The generated challenge ID, family, prompt and ground truth are stored server-side. The image file name is not trusted as the answer.
9. For uploads, the image-family classifier recognizes only the documented synthetic grammars: text, arithmetic, or the 3×3 color/shape grid. Anything else is unsupported or ambiguous.
10. The selected family solver processes the image. Text uses template matching; arithmetic recognizes a short expression and calculates it; image-grid uses color/shape analysis and cell coordinates.
11. If a server-issued challenge ID exists, the predicted answer is compared to server-stored ground truth. Otherwise the output is marked unverified, not a verified success.
12. The result, method, latency, confidence when meaningful, cost when known, and status are returned and recorded.

3.3 Workflow C: Official provider token verification

13. The user chooses Google reCAPTCHA, hCaptcha or Turnstile and loads that provider’s official test widget on a local fixture page.
14. The provider widget’s own callback supplies a response token. The UI labels this “token acquired — not yet verified”.
15. The token is sent to the FastAPI backend. The backend selects the provider-specific secret and official Siteverify endpoint.
16. The backend sends the token and secret to the provider. It returns verified only when the provider responds with success=true.
17. An actual provider rejection is reported as rejected. Network, timeout, configuration or malformed-response problems are reported as inconclusive.
18. The Playwright-assisted test opens the local fixture in Chromium and observes status attributes. For a visible checkbox widget, the candidate may complete the official test interaction in the opened window.

4. Module 01: URL-driven Playwright recognition

4.1 What the assessment asks

The assessment asks the application to identify challenge types and supported/unsupported cases, and specifically names Google reCAPTCHA, hCaptcha and Cloudflare Turnstile for controlled provider recognition. It does not require automatically submitting every Contact Us form.

4.2 What the UI does

The UI panel is titled “Inspect an Authorized Test Page”. It contains a target URL input, preset buttons for Turnstile/reCAPTCHA/hCaptcha, an “Inspect with Playwright” button, and a “Show Chromium window” checkbox. The prefilled local demo URL is a convenience for a reproducible test; it is not a cached result by design.

4.3 Function-level processing

Function / file	What it does	Why it exists

Function / file	What it does	Why it exists
validate_target_url() app/services/browser_inspector.py	Accepts only absolute HTTP/HTTPS URLs, rejects embedded credentials, normalizes hostnames with IDNA and checks the hostname against the configured allowlist.	Prevents arbitrary navigation and SSRF-style access to local/private resources.
_host_matches() app/services/browser_inspector.py	Matches exact hostnames and subdomains against the allowed host list; ignores “*” and “all”.	Avoids a wildcard turning the scanner into an unrestricted browser proxy.
_resource_allowed() app/services/browser_inspector.py	Allows page resources from the target allowlist, provider-resource allowlist and configured resource hosts; allows safe data/blob/about URLs.	Lets controlled provider widgets load required scripts while limiting unrelated external requests.
inspect_authorized_url() app/services/browser_inspector.py	Launches Chromium, loads the URL, records requests/failures/critical HTTP responses/page errors, scrolls to trigger common lazy-loaded content, traverses accessible frames, collects DOM snapshots and form metadata, runs recognition, saves screenshot and returns a structured report.	The main live webpage-recognition workflow.
detect_provider_from_html() app/services/recognition.py	Searches rendered markup, script/iframe URLs, attributes, frame URLs and resource URLs for supported/unsupported provider markers.	Centralizes provider recognition so it can be tested independently of a live browser.
_clean_report_url() / _redact_url_for_detection() app/services/browser_inspector.py	Remove credentials, query values and fragments from display/report URLs; preserve only selected safe query parameter names for marker detection.	Avoids leaking URL secrets into reports.
_inspection_result_error() app/services/browser_inspector.py	Builds a consistent failure response when navigation cannot be completed.	Prevents exceptions from becoming unstructured or misleading results.
_safe_browser_error_code() app/services/browser_inspector.py	Converts browser exceptions to a short safe error identifier.	Avoids exposing raw browser call logs or sensitive URLs.
browser_inspect() app/main.py	FastAPI route POST /api/v1/browser/inspect. Calls the inspector, maps allowlist errors to HTTP 403, missing Chromium to HTTP 503 and unexpected inspection errors to HTTP 502, then logs a summarized attempt.	Connects the UI to the Playwright service and creates a measurable inspection record.

4.4 Provider recognition outcomes

Outcome	Meaning	How the app responds
recaptcha	Supported Google marker found.	Reports provider and variant where detectable (v2/unspecified, v2 invisible, or v3 signal); next action is an official owned/test flow.
hcaptcha	Supported hCaptcha marker found.	Reports hCaptcha and points to its official test/verification flow.
turnstile	Supported Cloudflare Turnstile marker found.	Reports Turnstile and points to its official test/verification flow.
unknown / unsupported	Known unsupported provider marker or generic CAPTCHA-like marker found.	Does not guess or claim support; recommends manual review.
multiple	Markers for more than one supported provider found.	Marks the result ambiguous and requests manual review.
none	No supported provider marker is found after an adequate inspection.	This is only a “no supported marker” outcome; it is not proof that the site has no anti-bot control of any kind.
inconclusive_*	Navigation, HTTP, critical resources, frame collection, script errors or absence of a form made the inspection incomplete.	Explicitly says not to interpret the scan as CAPTCHA-free.

4.5 Why no contact form is submitted

The PDF explicitly excludes contact-form automation and bulk website submissions. For that reason, Module 01 collects form structure only; it does not fill name/email/message fields, click submit, collect provider response tokens, or attempt to bypass a live production challenge. This keeps the project aligned with the written acceptance conditions.

What “URL-driven” means here

It means the user supplies one authorized URL and Playwright renders that page to inspect its DOM/resources. It does not mean the application is authorized to crawl every CSV URL, submit forms, or pass any CAPTCHA on the public internet.

5. Module 02: Synthetic CAPTCHA identification and solving

5.1 What the assessment asks

The brief requires at least one custom OCR/computer-vision/machine-learning/other solver for a synthetic or candidate-controlled dataset, with solution and confidence where applicable. Recognition must distinguish supported challenge types from unsupported ones. The application implements three distinct controlled synthetic families to make the custom solving demonstration broader than the minimum.

5.2 The three synthetic challenge families

Family	Input task	Implementation method	Output
synthetic_text	Transcribe a five-character colored/rotated text string.	OpenCV grayscale/threshold/morphology/connected components; normalized glyph masks; cached multi-font templates; Intersection-over-Union style similarity matching for each fixed slot.	Five-character text, per-character match scores and average template-similarity score.
synthetic_arithmetic	Read a digit/operator/digit expression and calculate it.	Uses the same fixed-slot glyph recognition with a restricted arithmetic character set, then applies + or multiplication.	Expression, calculated answer and glyph similarity. It is not a general math OCR engine.
synthetic_image_grid	Select all 1-based cells matching a prompt such as a named color and shape in a 3×3 grid.	Pillow-controlled image generator; OpenCV HSV saturation mask, contour extraction, hue-based color classification, polygon/circularity-based shape classification and cell indexing.	Comma-separated matching cell positions plus per-cell color/shape classifications.

5.3 Generator and solver functions

Function / file	Functionality	Reason / design choice
generate_text_challenge() app/services/synthetic.py	Creates a five-slot text CAPTCHA, known answer and unique challenge ID.	Controlled labelled image for repeatable solver evaluation.
generate_arithmetic_challenge() app/services/synthetic.py	Creates a visual three-glyph expression (digit, operator, digit), answer, ID and prompt.	Adds a distinct reasoning step after visual transcription.
_background() / _place_glyph() / make_glyph_mask() app/services/synthetic.py	Creates textured/colored backgrounds and places rotated, colored glyphs using available fonts.	Introduces controlled visual variation while keeping a known ground truth.
generate_image_grid_challenge() app/services/image_grid.py	Creates a 3×3 grid with one shape per cell using six known colors and three shapes; returns prompt, correct cell numbers and challenge ID.	Adds a spatial visual-classification task, not another text variant.
_decode() / _normalize() / _template_bank() app/services/ocr.py	Decodes and validates images, normalizes glyph masks, builds a cached template bank for configured fonts/sizes/rotations.	Keeps processing local and avoids a separately installed OCR executable or downloaded model.
_extract_slot() / _recognize_fixed_slots() app/services/ocr.py	Extracts the largest glyph component in each fixed slot and compares it against the template bank.	Works because the bundled text/math generator uses a known fixed-slot grammar.
solve_text_image() app/services/ocr.py	Solves the text family and returns extracted characters and per-character similarity.	The core custom solver required by the assessment.
solve_arithmetic_image() app/services/ocr.py	Recognizes the three-character expression and computes addition or multiplication.	Separates image transcription from the simple arithmetic operation.
_parse_prompt() / _classify_color() / _classify_shape() app/services/image_grid.py	Parses supported prompt vocabulary, maps HSV hues to six color labels, classifies contour geometry.	Converts image observations to structured color/shape labels.
detect_image_grid_layout() / solve_image_grid_challenge() app/services/image_grid.py	Checks whether an image resembles the controlled 3×3 geometry layout; then returns target cell indices for the parsed prompt.	Provides a distinct grid-family recognition and solving path.

Function / file	Functionality	Reason / design choice
detect_synthetic_image_type() app/services/synthetic_detection.py	Checks grid layout first, then compares text and arithmetic template candidates; reports unsupported/ambiguous if evidence is insufficient or candidates are too close.	Separates challenge identification from solving and prevents forced guesses.
detect_synthetic_image() app/main.py	POST /api/v1/detect/synthetic-image. Validates upload size/type and invokes the automatic family classifier.	Allows an evaluator to upload an image and identify the supported family before selecting a solver.
generate_synthetic() app/main.py	POST /api/v1/synthetic/{family}/generate. Generates a selected family, stores answer/type/prompt/ID in SQLite and returns the PNG with challenge metadata in response headers.	The correct answer remains server-side rather than being trusted from a filename or user form field.
solve_synthetic() app/main.py	POST /api/v1/solve/{family}. Validates family, file, size, prompt and solver engine; runs the family solver; optionally invokes Gemini text fallback; looks up the stored challenge answer; returns exact-match/failed/unverified outcome and records metrics.	Orchestrates the full custom solving workflow and prevents unsupported uploads from being counted as confirmed successes.

5.4 Confidence and exact-match interpretation

For text and arithmetic, the OpenCV result contains a template-similarity score derived from glyph-template overlap. This score is not a statistically calibrated probability that the answer is correct. The grid solver currently returns confidence=None because its deterministic color/shape classification does not produce a calibrated probability. Gemini also returns confidence=None rather than inventing a score.

For generated challenges, the backend has a server-issued challenge ID and stored expected answer, so it can compare the predicted answer to the ground truth. For an arbitrary uploaded image without a valid challenge ID, the app can display a prediction but labels it unverified and does not count it as a successful exact match. This distinction is essential for honest accuracy reporting.

5.5 Supported synthetic scope — and limits

The grid family uses a fixed 420×420, 3×3 layout, one filled geometric object per cell, six named colors (red/orange/yellow/green/blue/purple), and three shapes (circle/square/triangle). The arithmetic family uses a fixed three-glyph expression with digits and + or x. The text family uses a five-slot alphanumeric format. The benchmark validates these generator grammars; it does not demonstrate arbitrary handwriting OCR, arbitrary photo recognition, or production provider challenge solving.

6. Module 03: Official provider token acquisition and verification

6.1 What the assessment asks

The PDF requires an official token-acquisition and server-side-validation demonstration using provider test keys or a candidate-controlled website. It explicitly calls for missing, invalid, expired and reused token handling, and structured responses that do not expose secret keys or sensitive tokens in logs.

6.2 What the module does

The UI lets the user select one of three providers, load its official test widget, receive the provider callback token, and then call the backend verification endpoint. The callback alone is never displayed as a successful verification. The UI explicitly says “TOKEN ACQUIRED — NOT YET VERIFIED” until the backend obtains a successful Siteverify response.

Function / file	What it does	Why it matters
_site_keys() / _provider_config() app/main.py	Builds a map of public site keys and provider script URLs for the local demo widgets.	The public site key is needed in the browser; provider secret keys are not sent to the frontend.
provider_demo() app/main.py	GET /demo/contact/{provider}; renders the local test form fixture with the chosen provider site key and JavaScript URL.	Creates a predictable candidate-controlled page for the official provider widget.
demo_contact.html JavaScript app/templates/demo_contact.html	Loads/render provider widget; callback sends token to `/api/v1/providers/{provider}/verify`; records only status attributes; handles expiry/error callbacks. The local form stays disabled until verification and never submits external form data.	Demonstrates real widget callback and server verification while avoiding token logging and external submissions.
loadProvider() / waitFor() app/static/app.js	Loads the provider script if necessary and waits for its render function to become available.	Supports the manual test-widget workflow.

Function / file	What it does	Why it matters
verifyProviderToken UI handler app/static/app.js	Sends the in-memory token to the backend when the user clicks Verify Token and renders the returned status.	Separates token acquisition from verification.
verify_provider_token() app/services/providers.py	Maps provider name to the official endpoint; validates token presence/length and secret configuration; sends provider-specific HTTP request; parses success/errors/hostname/timestamp; maps error codes; catches timeout/unavailable/invalid response.	Implements official server-to-server verification.
verify_token() app/main.py	POST /api/v1/providers/{provider}/verify. Selects site key/secret, calls provider adapter, classifies verified/rejected/inconclusive, records attempt and returns a structured response.	Central backend route used by both manual widget and local E2E fixture.
run_provider_widget_test() app/services/browser_inspector.py	Launches Chromium on the local provider fixture; waits for widget initialization, token callback status, and server verification status; observes only DOM status attributes, not the raw token; returns stage-specific errors.	Provides a Playwright-assisted end-to-end integration demonstration.
playwright_provider_test() app/main.py	POST /api/v1/browser/test-provider/{provider}. Runs the E2E helper, records success/failure/inconclusive distinctly and returns the timings/status.	Makes the provider flow repeatable from the dashboard.

6.3 Token states and failure cases

State / error	Meaning	Expected treatment
token acquired	Provider widget callback produced a token.	Not verified yet; send to backend.
verified	Provider Siteverify returns success=true.	Only this state is genuine success.
rejected	Provider response explicitly rejects token or returns an invalid/expired/reused/missing response code.	Display mapped provider error; do not call it a network timeout.
inconclusive	Timeout, network failure, bad provider response, or missing/invalid secret configuration stops verification from completing.	Do not claim the provider rejected the token; investigate configuration/connectivity.
awaiting_widget_interaction	Widget loaded but no token callback arrived before interaction timeout.	Complete the visible official test widget if the provider requires it, then retry with a fresh token.
widget_load_timeout	Provider script/widget did not initialize.	Check internet access, DNS, resource allowlist and site key.

6.4 Why there is no arbitrary URL field in the token validator

The token validator is not a page scanner. It is a provider verification adapter. Module 01 accepts an authorized URL because its job is to inspect the rendered page. Module 03 already knows the local test fixture route and provider test site key; its input is the provider choice and then the token produced by that widget. A production integration would configure an owned site's site key/secret pair in environment settings and embed the official widget on that owned site.

6.5 Why token retry is deliberately avoided

After a network timeout the provider may already have consumed the one-time token, even if the response did not reach the app. Automatically resending the same token could produce a duplicate/replay error and confuse the result. The implementation therefore does not automatically retry the same token after an uncertain timeout; it reports an inconclusive state and requires a fresh widget token.

7. Module 04: CSV import, batch evaluation, metrics and dashboard

7.1 The interviewer's CSV

The supplied `USA_Ecommerce_Platforms_200_Contact_Pages.csv` contains 200 contact-page records. The current app imports it as a reference dataset and displays its company/site/platform/CAPTCHA metadata. It does not automatically visit each website. The known distribution is:

CSV label	Records	Share
reCAPTCHA	103	51.5%
Invisible reCAPTCHA	30	15.0%
Turnstile	15	7.5%

CSV label	Records	Share
hCaptcha	6	3.0%
none	46	23.0%
Total	200	100%

This data is not the synthetic CAPTCHA image corpus. It is metadata about contact pages and their supplied labels. The app uses it for import, searching, pagination, distribution reporting and contextual reference. The CAPTCHA solver benchmark uses a separate synthetic manifest and corresponding generated images.

7.2 CSV import functions

Function / file	What it does
DatasetImportError app/services/dataset_import.py	Carries a stable error code and human-readable message for CSV parsing failures.
_normalize_header()	Normalizes case, spaces, underscores and BOM so known header aliases can be matched.
_decode_csv()	Attempts UTF-8 with BOM, UTF-16 and Windows-1252 decoding.
parse_dataset_csv()	Uses DictReader, maps aliases to canonical fields, checks required columns, skips empty rows, validates company/contact-page values and enforces a max row count.
replace_imported_sites() app/core/database.py	Replaces imported rows transactionally: if insertion fails, the delete is rolled back.
import_dataset() app/main.py	POST /api/v1/dataset/import. Validates `.csv` extension and byte limit, parses, stores, and returns imported count + distribution.
lifespan() app/main.py	Initializes the database and seeds the supplied CSV on startup only when the imported-sites table is empty; warms the solver template caches.

7.3 Dataset search and reporting

The dashboard table uses `/api/v1/dataset/rows` with search, page size and offset. `/api/v1/dataset/summary` returns total records and CAPTCHA-label distribution. The attempt dashboard uses `/api/v1/metrics`, backed by the `attempts` table, to aggregate total, successful, failed, unverified and inconclusive attempts, success rate, average/p95 latency, known estimated cost and the count of attempts with unknown cost.

7.4 Batch synthetic benchmark

The synthetic benchmark is independent of the interviewer's 200-row CSV. The synthetic manifest is `data/synthetic_dataset/labels.csv` and has 300 rows: 100 `synthetic_text`, 100 `synthetic_arithmetic`, and 100 `synthetic_image_grid`. `scripts/generate_synthetic_dataset.py` creates the PNG files and ground-truth manifest. `scripts/evaluate_dataset.py` selects samples in a balanced way, solves each, compares exact outputs, records mismatches, calculates per-family accuracy/average/p95 latency, overall accuracy/latency, and text character accuracy (for text family only).

The UI button "Run Three-Family Benchmark (300 Samples)" calls POST `/api/v1/dataset/evaluate-synthetic?samples=300`. `scripts/benchmark.py` is a convenience CLI that can regenerate then evaluate. The benchmark result is saved to `artifacts/dataset_evaluation_latest.json`.

Important packaging detail

The clean submission ZIP includes the 300-row `labels.csv` manifest and the generator, but deliberately excludes the generated PNG files (the images folder contains only `.gitkeep`). If the evaluator runs the benchmark and the images are missing, `evaluate_dataset.py` regenerates the reproducible corpus from the manifest/default seed. To show individual benchmark images, run the generator command before the demo.

8. Function-by-function code reference

This section maps the important functions to their source files. Private helper functions are included where they perform non-trivial processing, validation or security work. The UI event handlers are listed separately after the Python modules.

8.1 Application entry point — app/main.py

Function	Responsibility
lifespan()	On startup: initialize SQLite; seed the 200-row supplied CSV if the imported-sites table is empty; warm up OpenCV template caches. On shutdown: exits the FastAPI lifespan.
home()	Render the dashboard template with app name and public provider site keys.
health()	Report Python version, Playwright package availability, Chromium binary availability/readiness, OCR engine, target allowlist and wildcard status.
public_config()	Expose public provider site keys to the browser; never expose provider secrets.
detect_html()	Accept supplied HTML and run provider-marker recognition without launching a browser.
detect_synthetic_image()	Validate an uploaded image and auto-identify one of the three supported synthetic families.
browser_inspect()	Validate and orchestrate Module 01; translate allowlist/browser errors into structured HTTP errors; write attempt metrics.
playwright_provider_test()	Start the local provider E2E test and record success, rejection or inconclusive status.
provider_demo()	Render the local demo page for one selected provider.
generate_synthetic()	Generate one selected text/arithmetic/grid challenge, save ground truth and return image bytes plus challenge metadata headers.
solve_synthetic()	Validate and solve a synthetic upload; optionally use Gemini fallback for text only; compare to stored answer; return exact-match/unverified result and log metrics.
verify_token()	Call official server-side provider verification and classify verified/rejected/inconclusive.
metrics()	Return aggregated attempt and latency/cost metrics.
dataset_summary()	Return CSV record count and CAPTCHA distribution.
get_dataset_rows()	Search and paginate imported records.
evaluate_synthetic_dataset()	Run the balanced synthetic dataset benchmark in a worker thread.
import_dataset()	Upload, validate and transactionally replace the contact-page CSV dataset.

8.2 Configuration — app/core/config.py

Function / setting	Responsibility
Settings	Defines application name/environment, database/artifact paths, upload/CSV limits, provider/browser timeouts, base URL, Chromium path, allowed hosts/resource hosts, official provider test credentials and optional Gemini parameters.
_parse_hosts()	Splits and normalizes comma-separated hostname settings; deliberately discards wildcard entries.
allowed_hosts / resource_hosts	Expose the normalized target-host and provider-resource allowlists.
unsafe_wildcard_configured	Indicates that a legacy wildcard appears in configuration so health/UI can warn; wildcard is not granted access.
get_settings()	Cached environment-backed Settings loader using `.env`.

8.3 Persistence — app/core/database.py

Function	Responsibility
__init__()	Stores configured SQLite database path.
_connect()	Creates a configured SQLite connection, enables foreign keys/busy timeout, commits or rolls back transaction, and always closes the connection.
initialize()	Creates/migrates attempts, imported_sites and synthetic_challenges tables; adds indexes; corrects legacy Gemini metrics that were previously stored as fabricated confidence/$0.
add_attempt()	Persists timestamp, provider, challenge type, method, status, confidence, latency, known/unknown cost and error code.
replace_imported_sites()	Transactionally replaces CSV reference records.
save_synthetic_challenge()	Stores server-side challenge ID, answer, family, prompt and timestamp.
get_synthetic_challenge() / get_synthetic_answer()	Looks up stored ground truth and prompt by challenge ID.
dataset_summary()	Aggregates imported-site total and CAPTCHA label distribution.
count_imported_sites() / get_imported_sites()	Search, count, sort/paginate and return safe metadata columns.
metrics()	Aggregates attempt counts/statuses, success rate, average/p95 latency, known costs and unknown-cost count.

8.4 Browser inspection — app/services/browser_inspector.py

Function	Responsibility
_host_matches()	Matches a host to exact allowed entries or subdomains; excludes wildcard grants.
_clean_report_url()	Strips credentials, query values and fragment from URLs displayed in reports.
_redact_url_for_detection()	Retains only safe query parameter names with redacted values for marker inspection.
validate_target_url()	Enforces HTTP/HTTPS, valid host, no embedded username/password, and authorized host allowlist.
_resource_allowed()	Applies page/provider/resource host rules to browser requests.
_safe_browser_error_code()	Returns safe network/browser error identifiers.
_inspection_result_error()	Returns a consistent inconclusive navigation report.
inspect_authorized_url()	Main Playwright page-inspection routine: launch browser/context, navigation, resource tracking, scrolling, frame/DOM/form metadata capture, recognition, warning classification, screenshot and result object.
run_provider_widget_test()	Playwright-assisted official test widget sequence; observes local page state through DOM attributes, waits for user interaction when required, and distinguishes widget/token/Siteverify stages.

8.5 Recognition/import/provider/solver service modules

File and function(s)	Responsibility
app/services/recognition.py — detect_provider_from_html()	Provider rule engine: supported provider marker strings, known unsupported provider markers, multiple-provider ambiguity, generic CAPTCHA marker and no-marker outcomes.
app/services/dataset_import.py — DatasetImportError, _normalize_header(), _decode_csv(), parse_dataset_csv()	CSV errors, header normalization/aliases, encoding fallback, required-field and row-count validation.
app/services/synthetic.py — _font_candidates(), available_fonts(), make_glyph_mask(), _background(), _place_glyph(), _generate_slot_image(), generate_text_challenge(), generate_arithmetic_challenge()	Builds reproducible, visually varied text/math synthetic images and corresponding ground truth.
app/services/ocr.py — _decode(), _normalize(), _template_bank(), _extract_slot(), _recognize_fixed_slots(), solve_text_image(), solve_arithmetic_image(), warmup_solver()	Image validation, glyph normalization, cached template generation, slot extraction, template similarity recognition and arithmetic calculation.

File and function(s)	Responsibility
app/services/image_grid.py — _draw_shape(), generate_image_grid_challenge(), _parse_prompt(), _classify_color(), _classify_shape(), detect_image_grid_layout(), solve_image_grid_challenge()	Generates 3×3 geometric grids; parses prompt; uses HSV hue and contours to identify color/shape and select matching 1-based cells.
app/services/synthetic_detection.py — detect_synthetic_image_type(), _unsupported_result()	Recognizes the controlled text/arithmetic/grid image grammars and returns unsupported/ambiguous rather than guessing.
app/services/providers.py — _normalise_provider_errors(), _map_provider_errors(), verify_provider_token()	Provider error-code normalization and official HTTP Siteverify calls. It does not log or return raw token/secret.
app/services/vision_llm.py — _image_mime_type(), _safe_model_name(), solve_with_vision_llm()	Optional Gemini image transcription for synthetic text only; validates the response format; reports no fabricated confidence and cost unknown/estimated depending on billing/usage data.

8.6 CLI scripts

Script / function	Responsibility
scripts/generate_synthetic_dataset.py — _family_counts(), _remove_managed_images(), generate_dataset()	Creates a balanced deterministic image corpus and labels manifest. The generator controls the count/seed and removes only its own managed image filename patterns.
scripts/evaluate_dataset.py — _edit_distance(), _percentile95(), _balanced_select(), _solve(), run_evaluation()	Loads the manifest/images, regenerates the default corpus if missing/legacy, selects balanced rows, runs each family solver, compares exact match, calculates family/global metrics and writes JSON report.
scripts/benchmark.py — main()	Convenience command to optionally regenerate the dataset then run evaluation and print summary.
scripts/import_csv.py — main()	Command-line route to parse and import the supplied or another compatible CSV into the project database.

8.7 Frontend JavaScript functions — app/static/app.js

Function / handler	Responsibility
busy()	Disables an action button while a request runs and restores its label afterwards.
escapeHtml()	Escapes dynamic strings before inserting them into HTML to reduce injection risk in rendered results.
createDetailsJson()	Shows a collapsible technical JSON response for debugging/evaluator inspection.
renderInspectResult()	Formats inspection success, unsupported, inconclusive and error states; shows provider, challenge, form metadata, warnings and details.
renderSolveResult()	Formats exact-match, mismatch and unverified solver responses; shows answer, method, similarity/confidence if available, latency and cost notes.
renderVerifyResult()	Formats verified/rejected/inconclusive provider response states.
renderBatchBenchmarkResult()	Displays family-by-family batch benchmark results and aggregate latency/accuracy.
safeHttpUrl()	Validates/normalizes displayed web URL values used by the dataset UI.
loadDatasetRows() / renderDatasetSummary()	Fetch and display searched/paginated CSV records and distribution cards.
refresh()	Refreshes metrics, health state and dataset summary.
waitFor() / loadProvider()	Waits for provider JavaScript globals and loads the selected official widget script.
UI event handlers	Connect Inspect, Generate, Solve, Identify CAPTCHA Type, Upload Solve, Load Widget, Verify Token, Playwright Provider Test, CSV Import, Search/Pagination and Batch Benchmark buttons to the corresponding API routes.

9. Assessment traceability matrix and score mapping

The following matrix compares every scored requirement with the code path and the evidence that should be shown. “Implemented” means the relevant source path exists. It does not automatically mean a live internet/browser test has been completed on the candidate’s computer.

Criterion / weight	Assessment asks	Implementation path	Status / demo evidence
Recognition / solving accuracy — 30%	Identify supported/unsupported challenge type; recognize providers; solve custom synthetic challenge and report accuracy/confidence where applicable.	Module 01: recognition.py + browser_inspector.py. Module 02: synthetic_detection.py + ocr.py + image_grid.py + ground-truth DB.	Implemented for three controlled synthetic families and three named provider markers. Demo each family and provider; show unsupported/inconclusive outcomes. Template similarity is not calibrated probability.
Verification-token handling — 20%	Official provider token acquisition and server-side validation; missing/invalid/expired/reused; safe structured response.	demo_contact.html + app.js callback + providers.py + main.py verify_token() + local Playwright E2E.	Implemented in code using official test widgets/test credentials. Live provider network success still requires candidate-side verification. Show missing/invalid/reused/expired where deterministic test responses allow.
Custom AI or authorized API integration — 20%	Custom OCR/CV/ML or permitted low-cost API.	OpenCV/NumPy/Pillow solver stack; optional Gemini remote vision fallback for text only; Siteverify APIs for three providers.	Custom CPU CV implementation present; Gemini optional, not required. No paid CAPTCHA-solving service integrated. Show local solver and, if configured, Gemini fallback.
Reliability / error handling — 15%	Sensible timeouts, structured errors, unsupported handling and safe secrets/tokens.	Pydantic request validation; file/CSV limits; Playwright URL/resource allowlist; HTTPX timeouts; status classification; SQLite migration and tests.	Implemented in source. Demonstrate malformed image, unsupported type, allowlist rejection, missing token and timeout/inconclusive state.
Interface / reporting — 10%	Upload/select, trigger, display; totals, success/fail, time, challenge, method/provider and cost if used.	index.html + app.js + main.py metrics and data routes + Database.metrics().	Implemented. Show dashboard, result cards, CSV distribution, attempts and benchmark report.
Code quality / documentation — 5%	Readable modular structure and setup/env docs.	app/core, app/services, scripts, tests, README.md, BENCHMARK.md, .env.example, setup/run scripts.	Implemented as source/docs. Candidate should run setup and tests from clean extraction and exclude .env.

10. Bonus feature comparison

Bonus item in PDF	What this project implements	Assessment of coverage / limits
Multiple supported challenge types or providers	Three synthetic families (text, arithmetic, image-grid) and three official providers (reCAPTCHA, hCaptcha, Turnstile). Unsupported provider markers are also recognized.	Implemented. Synthetic image-grid means controlled geometric shapes, not arbitrary photo selection. Provider recognition/verification is not provider challenge bypass.
Fallback handling if primary method fails	OpenCV is primary. In `engine=auto`, if text OCR returns no result or similarity falls below configured threshold and Gemini key is configured, Gemini vision is called. Gemini can also be selected explicitly for text family.	Partial/targeted fallback. It is only for synthetic text. Arithmetic/grid do not use Gemini fallback. It may increase latency and remote API quota/cost.
Batch processing and accuracy statistics for supplied synthetic dataset	300-sample balanced benchmark, 100 per family; exact-match accuracy, family metrics, mean/p95 latency, text character accuracy, mismatch samples and JSON output. UI/API/CLI entry points included.	Implemented for the project’s synthetic dataset. It does not batch-scan or submit the 200 third-party CSV contact pages. Those CSV records are metadata.
Reusable API endpoint for authorized application integrations	FastAPI endpoints for inspect, synthetic detect/generate/solve, provider verify, CSV import and metrics.	Implemented as reusable local/controlled API routes. The project is an assessment application, not a hardened public multi-tenant service; public deployment would need additional authentication, rate limiting and operational hardening.

11. Why the application deliberately does not do certain things

Not implemented / restricted behavior	Reason	What the application does instead
Automatically crawl all 200 CSV contact pages	The PDF says contact-form automation and bulk website submissions are not part of the assignment; the scope limits live testing to synthetic/local/owned/official provider environments.	Imports the 200 rows as reference metadata; supports one-at-a-time inspection only for allowlisted authorized hosts.

Not implemented / restricted behavior	Reason	What the application does instead
Submit Contact Us forms or enter user data	Explicitly outside written assessment scope and unnecessary to demonstrate recognition/token verification.	Reads form method/action/field count/types only; reports `forms_submitted: false`.
Harvest tokens from unrelated websites	Explicitly outside the acceptance scope and unsafe to treat third-party response tokens as test data.	Obtains tokens only from the official test widget on a local/candidate-controlled fixture; token is sent to the owned backend verifier.
Bypass every production CAPTCHA	The assignment asks for supported CAPTCHA workflows in an authorized environment, not a universal production CAPTCHA bypass.	Recognizes the three named providers; routes to official test verification. Unsupported/ambiguous types are reported for manual review.
Use one universal OCR/AI model for every CAPTCHA	Different challenge families require different input/solver logic; provider tokens are not solved by OCR.	Uses specialized text, arithmetic and shape-grid solvers; separate official provider verification.
Treat similarity as true probability	Template overlap is a heuristic score and not statistically calibrated.	Labels it as template similarity; grid/Gemini confidence remains null where no calibrated value exists.
Automatically retry a token after a verification timeout	The provider may have consumed the one-time token even if the response was lost.	Returns inconclusive and asks for a fresh token rather than replaying the same token.
Run GPU models or require a separate Tesseract executable	The candidate has CPU-only hardware and the assessment can be demonstrated with local CV.	OpenCV + NumPy + Pillow template matching; no GPU, Tesseract executable or model download required.
Open Chromium visibly for every scan	Headless browser mode is faster and normal for automation.	Module 01 is headless by default with a visible-browser option; provider E2E is visible by default for human checkbox interaction.

12. Technology, cost, configuration and security

12.1 Technology stack

Layer	Technology in the project	Reason
Backend/API	Python 3.12/3.13 x64, FastAPI, Uvicorn, Pydantic Settings	Typed API contracts, request validation, local web server and modular service routes.
Browser automation	Playwright Python + managed Chromium	Renders JavaScript pages and exposes DOM/frame/network information.
Computer vision	OpenCV-headless + NumPy + Pillow	Local image decoding, mask processing, template similarity, HSV and contour analysis.
HTML inspection	BeautifulSoup4	Parses HTML snapshots and searches tags/attributes/scripts.
HTTP provider integration	HTTPX	Calls official Siteverify endpoints with finite timeouts.
Frontend	Jinja2 templates + vanilla JavaScript + CSS	Lightweight dashboard without a separate frontend build chain.
Persistence	SQLite	Stores attempts, CSV records, and generated challenge ground truth.
Testing	Pytest + pytest-asyncio	Unit/API/regression testing.

12.2 Cost breakdown

Component	Cost treatment	Notes
OpenCV / NumPy / Pillow solvers	$0 external solver cost	Runs locally on CPU; dependency install still needs internet once.
Playwright + Chromium	$0 license/API usage for this local workflow	Browser binary download required during setup; local runtime.
SQLite / FastAPI / Python libraries	$0 license/API usage	Local software dependencies.
Official provider test widgets / test keys	$0 for published test flows	They demonstrate integration; they do not provide production anti-bot protection.

Component	Cost treatment	Notes
Optional Gemini fallback	Free or usage-based according to configured Gemini billing tier	Only applies to synthetic text. If paid, cost is estimated from usage metadata and configured rates; if usage data is absent, cost remains unknown. It can add network latency.
Paid CAPTCHA-solving API	Not used	The custom CV path satisfies the “at least one” custom-solver option; no paid solver service is needed.

12.3 Environment variables worth understanding

Variable	Purpose
ALLOWED_TARGET_HOSTS	Comma-separated local/owned/authorized hosts that Module 01 may navigate to. Wildcard values are ignored.
ALLOWED_RESOURCE_HOSTS / BROWSER_RESOURCE_HOSTS	Additional specific resource/CDN hosts needed by owned test pages and official provider scripts.
APP_BASE_URL	Base address used by the Playwright-assisted local provider fixture.
BROWSER_TIMEOUT_MS	Navigation/inspection timeout.
PROVIDER_WIDGET_TIMEOUT_SECONDS	Time allowed for the official widget script to initialize.
PROVIDER_INTERACTION_TIMEOUT_SECONDS	Time allowed for a widget callback, including user interaction where necessary.
PROVIDER_VERIFICATION_TIMEOUT_SECONDS / PROVIDER_TIMEOUT_SECONDS	Limits for backend Siteverify requests.
PROVIDER_TEST_HEADLESS	Whether provider E2E runs hidden; default false for visible manual test interaction.
MAX_UPLOAD_BYTES / MAX_CSV_BYTES	Upload limits for images and CSV.
GEMINI_API_KEY / GEMINI_MODEL / GEMINI_TIMEOUT_SECONDS	Optional Gemini text fallback settings.
GEMINI_FALLBACK_THRESHOLD	OpenCV template-similarity threshold that can trigger fallback in auto mode.
GEMINI_BILLING_TIER and input/output rates	Controls whether cost is shown as free-tier estimate, paid usage estimate, or unknown.

12.4 Security protections

- Only public provider site keys are returned to the browser. Provider secret keys stay server-side in environment settings.
- Raw response tokens and secret keys are not logged, persisted in attempts, or returned in reports.
- Module 01 validates HTTP(S), hostname allowlist and embedded credentials, and restricts external resource hosts.
- URLs displayed in reports are sanitized to remove credentials, query values and fragments.
- CSV and image uploads have type/size/row validation; database replacement is transactional.
- Generated challenge ground truth is stored server-side and is not trusted from user filenames or arbitrary upload metadata.
- SQLite connection lifecycle commits/rolls back and closes connections; test database/artifacts are isolated.

13. Dataset packaging, reproducibility and test evidence

13.1 What is in the final clean package

The final submission ZIP contains source code, `.env.example`, dependency pins, setup/run scripts, tests, the supplied contact-page CSV, and the synthetic `labels.csv` manifest. The `data/synthetic_dataset/images/` directory contains `.gitkeep` only. The packaging script deliberately excludes generated PNGs, databases, `.env`, `.venv`, caches, logs and screenshots. This is intentional to keep the source package clean and avoid shipping machine-specific or generated runtime state.

The evaluator can reproduce the 300 image corpus with:

```
.\.venv\Scripts\python.exe scripts\generate_synthetic_dataset.py --samples 300 --seed 20261112
```

Then evaluate it with:

```
.\.venv\Scripts\python.exe scripts\evaluate_dataset.py --samples 300
```

Or regenerate and benchmark in one command:

```
.\.venv\Scripts\python.exe scripts\benchmark.py --samples 300 --regenerate --seed 20261112
```

13.2 Documented benchmark results

The packaged BENCHMARK.md records a fixed-seed benchmark of 300 generated samples: 100 per family. Results in that file are 100/100 exact matches for each family and 300/300 overall, with average solver latency 25.22 ms (text), 10.61 ms (arithmetic), 4.67 ms (image-grid), 13.50 ms overall; overall p95 32.67 ms. It also records a separate 300-case synthetic family-identification check. These figures were measured in the build environment, not on every evaluator computer.

Benchmark interpretation

These results validate the solver against the project's own controlled generator grammar. They do not demonstrate that the solver can solve arbitrary real-world CAPTCHAs or production provider challenges. State that scope when presenting the numbers.

13.3 Test suite coverage

Test file	What it verifies
test_api.py	Health/home/API routes, CSV seed/import/distribution and related API behaviour.
test_browser_inspector_helpers.py	URL sanitization, resource allow checks, error/status helpers and inspection logic.
test_browser_security.py	Allowlist and URL security behaviour.
test_database.py	SQLite schema, migrations, attempt records and imported dataset operations.
test_providers.py	Token validation, provider error mapping, mocked provider responses and timeout/inconclusive distinction.
test_recognition.py	Provider marker recognition and unsupported/ambiguous cases.
test_synthetic_detection.py	Automatic classification of the controlled synthetic families.
test_synthetic_families.py	Generation and solver behavior across text, arithmetic and image-grid families.
test_synthetic_ocr.py	Text/math OCR processing and exact answer logic.
test_vision_llm.py	Gemini request construction, AQ/AIza header handling, response validation and error mapping (mocked network).

The packaged test directory contains 71 test functions. Automated tests are not a substitute for a live Windows Chromium scan or a real network call to each official provider. Before final submission, run the suite on the candidate PC and separately demonstrate the local provider widget and Siteverify flow.

Suggested technical explanation

"I separated the project into recognition, synthetic solving, and official verification because these are different problems. Playwright inspects an authorized rendered page and identifies provider markers without submitting the form. The custom solver handles three candidate-controlled synthetic families with CPU OpenCV. For Google, hCaptcha and Turnstile, the official widget issues the token and the backend validates it using the provider's Siteverify endpoint. The CSV is imported as reference metadata, while the synthetic benchmark measures our own controlled generators. Unsupported or inconclusive cases are reported rather than being falsely treated as solved."

Appendix A — Key project files at a glance

Path	Role
app/main.py	FastAPI application, lifespan, request models, all API routes and orchestration.

Path	Role
app/core/config.py	Environment-backed settings, host allowlist parsing and runtime configuration.
app/core/database.py	SQLite schema, migrations, persistence and metrics aggregation.
app/services/browser_inspector.py	Playwright navigation, form metadata, DOM/frame/resource inspection and local provider E2E.
app/services/recognition.py	Provider marker rule engine.
app/services/synthetic.py	Text/arithmetic image generation and glyph/background utilities.
app/services/ocr.py	CPU template matching for text and arithmetic.
app/services/image_grid.py	Synthetic 3×3 grid generation and CV solver.
app/services/synthetic_detection.py	Automatic identification of supported synthetic families.
app/services/providers.py	Official provider Siteverify HTTP requests and error mapping.
app/services/vision_llm.py	Optional Gemini fallback for synthetic text.
app/services/dataset_import.py	CSV parsing, column aliases and validation.
app/templates/index.html	Main dashboard and module panels.
app/templates/demo_contact.html	Local provider test fixture; the form does not submit externally.
app/static/app.js / style.css	UI event wiring, API calls, result renderers and visual styling.
scripts/generate_synthetic_dataset.py	Generate images and balanced labels manifest.
scripts/evaluate_dataset.py	Batch exact-match benchmark and latency metrics.
scripts/benchmark.py	Convenience regenerate + evaluate command.
scripts/import_csv.py	CLI CSV importer.
requirements.txt / requirements-dev.txt	Pinned runtime and test dependencies.
setup.ps1 / setup.bat	Create environment, install packages and Playwright Chromium.
run.ps1 / run.bat	Start the local FastAPI app.
package_submission.ps1	Create clean archive without secrets/runtime/generated images.
README.md / BENCHMARK.md	Setup, architecture, scope, endpoints, cost and reproducible benchmark notes.